

Object-oriented Programming 2 KU

Mapping Requests, Graphics2D, GeoTools and Path-Finding Algorithms (Routing)

(Basically, all the content for Assignment 2)

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

Mapping Requests and Graphics2D

Mapping Requests



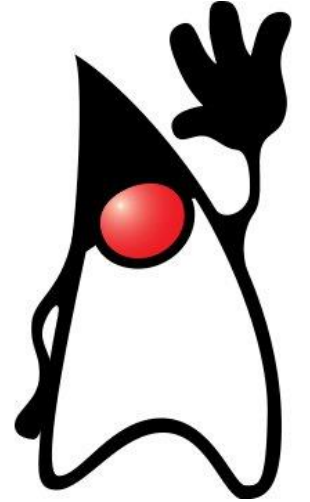
- Mapping Requests:
 - These requests are used to generate images of maps based on user inputs like coordinates and layers.
- Spring Boot & gRPC:
 - The server handles mapping requests via gRPC, using StreamObserver to respond with the generated image.
- Key Components:
 - MapRequest: Holds information about coordinates (X, Y, Z) and layers to render.
 - MapResponse: Encapsulates the image data and metadata (size, format).

Mapping Requests



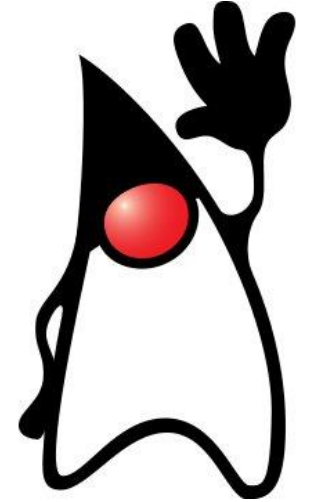
- Request Handling:
 - The method `mapRequest` listens for requests, logs the details, and generates a response.
- `MapLogger.backendLogMapRequest`:
 - Logs the incoming request details.
- `BufferedImage img = drawer.draw(request)`:
 - Uses the drawer object to draw the map based on the request.
- Image is then converted into bytes and sent as a response.
- gRPC Communication:
 - Uses `StreamObserver` to asynchronously send a `MapResponse` back to the client.

Graphics2D: Core of Java Image Drawing



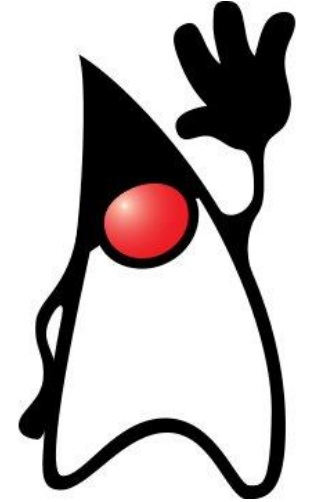
- Graphics2D: A Java API used for drawing 2D shapes, text, and images.
 - Provides fine control over geometric shapes and image manipulation.
 - Supports anti-aliasing, transformations, and color rendering.
- How it Works:
 - BufferedImage: The image that is drawn is stored in memory as a BufferedImage.
 - Drawing Operations: Calls to Graphics2D draw map elements like lines, polygons, or tiles.

Graphics2D: Key Operations in Mapping



- Core Drawing Elements:
 - Shapes: `drawLine()`, `drawRect()`, `fillPolygon()`.
 - Images: `drawImage()` can be used to overlay images or textures.
 - Text: `drawString()` for labeling map features.
- Coordinate System:
 - The map tiles or shapes are drawn based on X, Y, Z coordinates, which translate to pixel positions on the image.

Graphics2D: Image Generation Process



- Drawing the Map:
 - The map is drawn based on the MapRequest which contains the tiles, layers, or other visual elements.
- Converting Image to Byte Array:
 - Once the image is drawn (BufferedImage), it's written into a byte array using **ImageIO.write(...)**
 - This byte array is then included in the MapResponse, which is sent back to the client.

Introduction to GeoTools

GeoTools



- GeoTools: A powerful Java library for geospatial data manipulation and analysis.
- Key Capabilities:
 - Coverage calculations (geospatial analysis like areas and grids).
 - EPSG conversions (transformations between coordinate systems).
 - Routing (finding the shortest paths between locations).
 - Visualization (creating maps, rendering geometries).

GeoTools: Example Functionalities



- Coverage Calculations:
 - Handles raster data, allowing for complex geospatial analysis (e.g., terrain or environmental modeling).
- EPSG Conversions:
 - Supports thousands of coordinate reference systems via EPSG codes.
 - Allows conversion between global and local systems (e.g., from WGS84 to local projections).
- Visualization:
 - Tools for rendering geometries, maps, and styling features.

GeoTools: Routing Capabilities



- Routing:
 - Capable of calculating paths between two or more nodes in a graph.
 - Provides support for custom weighting of edges (e.g., by distance, travel time, etc.).
- Nodes and Edges:
 - Routing is based on graphs made up of nodes (points/locations) and edges (the connections between them).
 - Nodes and edges can be extracted from your spatial data.

GeoTools: Custom Weighting



- Custom Edge Weighting:
 - GeoTools allows users to define custom weights for edges in the routing graph.
 - Commonly, edge weight is based on geometry length or other properties.
- Applications:
 - Weight can be customized based on travel time, road quality, terrain difficulty, etc.

Shortest Path Algorithms

Shortest Path Algorithms



- Shortest Path Algorithms Examples:
 - Dijkstra's Algorithm:
 - Finds the shortest path from a start node to all other nodes by exploring all possible routes, guaranteeing the shortest one.
 - Example Code from GeoTools [here](#).
 - A*:
 - An improved version of Dijkstra, using heuristics to focus the search on the destination, making it faster in many cases.

Shortest Path Algorithms



Bae: Come over

Dijkstra: But there are so many routes to take and
I don't know which one's the fastest

Bae: My parents aren't home

Dijkstra:

Dijkstra's algorithm

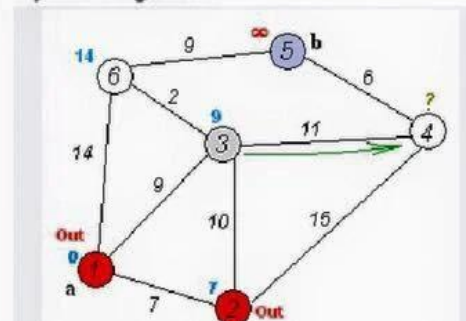
Graph search algorithm

Not to be confused with Dykstra's projection algorithm.

Dijkstra's algorithm is an **algorithm** for finding the **shortest paths** between **nodes** in a **graph**, which may represent, for example, road networks. It was conceived by **computer scientist Edsger W. Dijkstra** in 1956 and published three years later.^{[1][2]}

The algorithm exists in many variants; Dijkstra's original variant found the shortest path between two nodes,^[2] but a more common variant fixes a single node as the "source" node and finds shortest paths from the source to all other nodes in the graph, producing a **shortest-path tree**.

Dijkstra's algorithm



Dijkstra's Algorithm



- How Dijkstra's Algorithm Works:
 - Iteratively explores the nearest unvisited node, updating the shortest known path from the start node.
 - Guarantees that when a node is marked as "visited," the shortest path to it has been found.
- Use Cases:
 - Useful when calculating paths in a fully connected graph, where every node can reach every other node.

Dijkstra's Algorithm



```
// Reference to an already-built graph
Graph graph = ...;

// Source node
Node start = ...;

// Edge weighting based on geometry length
DijkstraIterator.EdgeWeighter weighter = (Edge e) -> {
    SimpleFeature feature = (SimpleFeature) e.getObject();
    Geometry geometry = (Geometry) feature.getDefaultGeometry();
    return geometry.getLength();
};

// Create and run DijkstraShortestPathFinder
DijkstraShortestPathFinder pf = new DijkstraShortestPathFinder(graph, start, weighter);
pf.calculate();

// Find paths to destinations
List<Node> destinations = ...;
for (Node destination : destinations) {
    Path path = pf.getPath(destination);
    // Handle the calculated path
}
```

■ [Code Source](#)

A* Algorithm



- How A* Works:
 - Like Dijkstra, it explores paths, but it uses a heuristic to estimate the distance from each node to the destination, allowing it to focus on the most promising paths.
- Faster than Dijkstra:
 - By focusing the search, A* reduces the number of nodes visited, making it faster for many real-world applications.

A* Algorithm



```
AStarShortestPathFinder pf = new AStarShortestPathFinder(graph, start, weighter);  
pf.calculate();  
  
for (Node destination : destinations) {  
    Path path = pf.getPath(destination);  
    // Handle the path  
}
```

Dijkstra vs. A* Algorithm



- Dijkstra:
 - Always finds the shortest path by exploring all possible routes.
 - More computationally expensive as it explores many nodes.
- A*:
 - Uses heuristics to guide the search.
 - More efficient in large graphs when you need to quickly find the shortest path to a specific destination.

Dijkstra vs. A* Algorithm



- Use Dijkstra:
 - When you need to calculate the shortest path to all nodes.
 - Works well when the graph is small, or the cost of computation isn't a major concern.
- Use A*:
 - When you only need to calculate the shortest path to a specific destination.
 - Preferable for large graphs or real-time applications, where efficiency matters.

For the assignment: Choose whatever you prefer 😊

Further Resources

Further Resources



- [Official documentation](#)
- [Baeldung](#)
- [Recording from Last year](#) (With introduction to Assignment 2)
- [Code Source](#) (from the slides)
- [Dijkstra's Algorithm](#)
- [A* Algorithm](#)

Object-oriented Programming 2 KU

Mapping Requests, Graphics2D, GeoTools and Path-Finding Algorithms (Routing)

(Basically, all the content for Assignment 2)

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)